



UNIVERSITY OF THESSALY

Sign Language Recognition system using Machine Learning

Student: Garyfallia Anastasia Papadouli | 03533
Supervisor: George Thanos

June 8, 2026

Contents

Summary	2
Literature Review	2
Overall Goal and Methodology	2
System Architecture	3
Data pipeline	3
Landmarking	4
Feature Extraction	5
Model structure	6
Linear Projection and Positional Encoding	6
Transformer Encoder Blocks with Pre-Layer Normalization	6
Masked Global Average Pooling and Classification Head	7
Implementation	7
Software stack	7
Project structure	7
Challenges and adaptations	8
Results	8
Hyperparameter optimisation	8
Model performance and convergence	10
Error Analysis and Confusion Review	10
Demonstration application	12
Appendix: Accuracy curves for different model configurations	12
Conclusion	14

Summary

Automatic Sign Language Recognition (SLR) plays a crucial role in bridging the communication gap between the deaf community and the hearing population. However, continuous and isolated word recognition poses significant computational challenges due to the complex spatial and temporal nature of visual signs. This project presents a lightweight, end-to-end architecture for Greek Sign Language (GSL) word recognition across a large vocabulary of **369 classes**. Moving away from computationally heavy RGB-based 3D convolutional networks, this project’s proposed pipeline utilizes **MediaPipe Tasks** to extract normalized landmarks from video frames. These spatial features are then modeled using a temporal **Transformer** network implemented in **PyTorch**. Evaluated on a publicly available, balanced GSL dataset, the system achieves a remarkable **68,3% Top-1 Accuracy** and **93.5% Top-5 Accuracy** in a strict **signer-independent** setup. The decoupled nature of the feature extraction combined with the structural optimization of the Transformer ensures high inference speed and low memory footprint, rendering the system highly suitable for real-time deployment on low-powered and mobile devices.

Literature Review

Numerous studies have focused on the temporal recognition of continuous signs, ranging from alphabets to individual words. For instance, [1] proposes a system for detecting numerical signs, the alphabet, and common expressions (e.g., "hello"), alongside a feature for user-defined custom signs, utilizing models built with **TensorFlow** in conjunction with **MediaPipe Hands** for real-time hand tracking. Moving beyond hand tracking, [2] investigates the integration of **MediaPipe Holistic** for complete pose estimation, deploying **LSTM** networks optimized via **TensorFlow Lite**. More recent approaches have extended this task into the domain of Natural Language Processing (NLP) for full sign-to-text translation [3, 4].

Regarding Greek Sign Language (GSL), several machine learning efforts have been made toward automated recognition. A milestone work by [5] introduced a large, publicly available GSL dataset. The authors evaluated both 2D (frame-by-frame) and 3D (spatio-temporal) neural networks, achieving an 89.74% accuracy for isolated sign recognition and a 6.10% Word Error Rate (WER) in signer-independent continuous recognition. While [5] focuses primarily on recognizing individual glosses in their original signing order, [4] implements an end-to-end translation system. This architecture reconstructs visual signals into grammatically correct spoken Greek text by utilizing geometric landmarks and trajectories extracted via **OpenPose** in combination with a **Transformer encoder-decoder** network.

Papadimitriou et al. (2025) from the **University of Thessaly** present the **largest publicly available Greek Sign Language dataset to date**, [6] collected for an educational platform and comprising continuous signing sequences across an extended vocabulary. Their recognition pipeline employs **MediaPipe** for skeletal landmark extraction combined with deep learning classifiers, establishing a strong baseline and a reusable benchmark for GSL research.[7]

Overall Goal and Methodology

The primary objective of this project is to develop a system that accepts a video file or a live camera feed as input and recognizes a continuous stream of signs in Greek Sign Language (GSL) in their

chronological order of signing. Based on recent advancements in the field, this work applies a similar pipeline logic to [2], utilizing the publicly available, balanced, and signer-independent Isolated GSL corpus created by the Electrical & computer engineering department of University of Thessaly. [6].

The methodology begins with extracting spatial landmarks from the video frames using **MediaPipe Tasks**, followed by a crucial normalization step to ensure invariant geometric representation. By leveraging **PyTorch** for temporal modeling via a **Transformer** architecture in conjunction with **MediaPipe’s lightweight feature extraction**, execution speed and portability are prioritized. This optimization minimizes the hardware requirements, rendering the system highly suitable for low-powered and mobile devices. As a future extension, a Small Language Model (SLM) could be integrated to receive the recognized stream of glosses, rearrange them, and infer the correct spoken-Greek grammatical structure, effectively transitioning the system from word recognition to full translation.

System Architecture

Data pipeline

The data pipeline serves as the foundational infrastructure of the system, responsible for ingestion, structural formatting, and sequence preparation. The primary objective of this pipeline is to ingest raw, high-resolution video streams and efficiently format them into **standardized, low-overhead numerical sequences** optimized for real-time model training and inference.

The SL-ReDu corpus [7], developed at the University of Thessaly, constitutes the largest Greek Sign Language dataset to date, comprising 36 hours of RGB-D video from 21 signers recorded under studio conditions (three times larger in both duration and number of signers than the previously largest GSL dataset, the ITI GSL corpus [5]). In this work, the **isolated sign recognition subset** is employed, consisting of **22,632** video samples across 369 sign classes, with each signer performing **3 repetitions per gloss**, and the recognition pipeline leverages MediaPipe Holistic for multi-modal landmark extraction across hand, pose, and face components.

To prevent redundant computation during iterative training cycles, the pipeline splits processing into an offline extraction phase and an online training phase.

During the offline phase, raw video sequences are passed sequentially through the landmarking architecture. The resulting spatial coordinates are serialized and stored as **NumPy binary files** (`.npy`). This caching mechanism drastically reduces the I/O bottleneck during the training phase, as the model **directly loads dense coordinate arrays** rather than reprocessing heavy pixel data from compressed video containers on every epoch.

Because natural sign language gestures vary in duration, the extracted feature arrays exhibit variable temporal lengths. To optimize computational throughput and minimize memory overhead, an empirical analysis of the dataset’s video lengths was conducted. Based on the distribution, a strict temporal cap of $T_{\max} = 90$ frames was implemented.

During mini-batch generation, sequences are processed as follows:

- **Truncation:** Outlier sequences exceeding 90 frames are truncated to the maximum threshold, preventing an artificial inflation of the tensor space due to rare, exceptionally long videos.
- **Padding:** Sequences shorter than 90 frames are post-padded with zero-vectors to reach the uniform length.

- **Masking:** A corresponding boolean attention mask is generated alongside the tensors, ensuring the downstream Transformer blocks ignore these padded padding tokens during self-attention calculations.

The batch size parameter is configured within the data loading pipeline external to the network architecture. Consequently, the final tensor delivered to the model input at each training iteration has a standardized shape of $(B, 90, 186)$, where B represents the dynamic batch size and 186 signifies the flattened feature coordinates.

Landmarking

Landmarking is the process of extracting specific coordinates in 3D space from 2D images or video frames to map the structural outline of an object or body part into a compact, numerical format. By replacing raw, heavy pixel data with a finite set of geometric coordinates, landmarking dramatically slashes the dataset’s footprint meaning that it strips away background noise and unnecessary pixels, leaving only the essential mathematical skeleton. Utilizing **MediaPipe**, Google’s open-source framework, this process becomes highly streamlined. By leveraging its **pre-built computer vision Task APIs for pose and hand estimation**, developers can achieve real-time tracking with minimal computational overhead.

The hand landmarker model detects **21 distinct** hand-knuckle coordinates within the identified hand regions. Concurrently, the pose landmarker tracks 33 body landmark locations, capturing the overall skeletal structure. However, because the source dataset consists of videos showing only the signers’ upper bodies, several landmarks were redundant or irrelevant to the task. Consequently, landmarks from the waist down were discarded, as well as the wrist and hand landmarks, which are already tracked with higher precision by the dedicated hand landmarker model. While integrating a separate face landmarker task was initially considered, the subset of facial landmarks already provided by the pose estimation model proved sufficient to capture meaningful facial context without adding computational overhead.

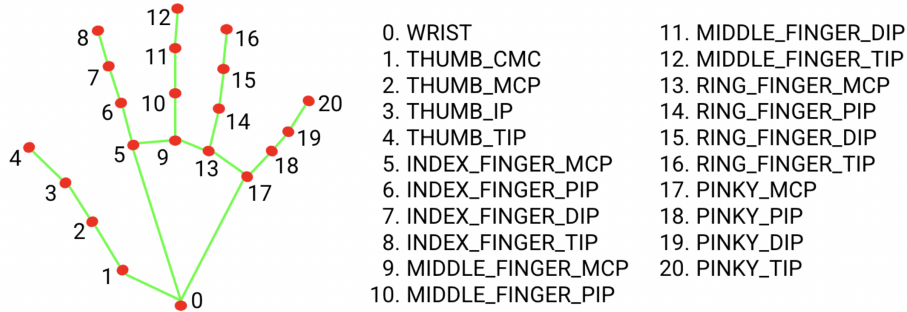


Figure 1: The 21 default hand landmark locations

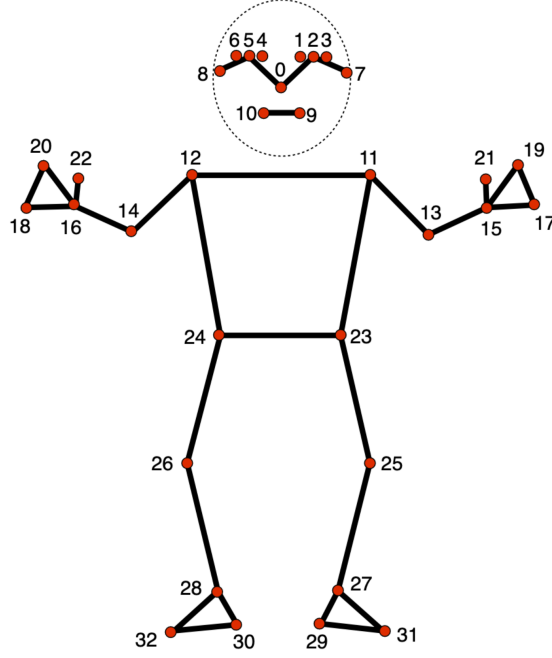


Figure 2: The 33 default pose landmark locations

Feature Extraction

Through the combination of the MediaPipe hand and pose landmarker models, a specific subset of keypoints is extracted to form the final feature vector for each video frame.

While the hand landmarks consist strictly of three-dimensional spatial coordinates (x, y, z) , the pose landmarks retain an additional **visibility score** (v). This fourth dimension indicates the probability that a specific joint is visible rather than occluded by the signer’s body, providing critical structural context for sign language recognition tasks.

To maximize computational efficiency and avoid processing irrelevant data, the total landmark set was pruned to capture only the upper body. The final feature selection per frame consists of:

- **Left Hand:** 21 landmarks with 3 spatial coordinates each ($21 \times 3 = 63$ features)
- **Right Hand:** 21 landmarks with 3 spatial coordinates each ($21 \times 3 = 63$ features)
- **Upper Body Pose:** 15 selected landmarks with 4 metrics each (x, y, z , and visibility), resulting in $15 \times 4 = 60$ features.

This brings the configuration to a total of 57 tracked landmarks per frame. The overall dimension of the flattened feature vector (F) for a single frame is calculated as follows:

$$F = (21 \times 3) + (21 \times 3) + (15 \times 4) = 186 \text{ features} \quad (1)$$

For each video sequence, these 186 features are flattened into a single 1D array per frame, serving as the standardized input for the subsequent classification network.

Model structure

The sequence-based classification network is constructed around a Transformer Encoder topology, designed to capture long-range temporal dependencies across the sign language frames. The model processes input tensors of shape (B, T, D_{in}) , where B represents the batch size, T denotes the temporal sequence length, and $D_{\text{in}} = 186$ corresponds to the raw spatial landmark features.

Linear Projection and Positional Encoding

Because raw coordinate dimensions do not inherently match the latent space of the network, the D_{in} -dimensional feature vector of each frame is first mapped to a higher-dimensional embedding space (d_{model}) via a linear projection layer:

$$X_{\text{proj}} = X \cdot W_{\text{proj}} + b_{\text{proj}} \quad (2)$$

where $W_{\text{proj}} \in R^{D_{\text{in}} \times d_{\text{model}}}$ and $b_{\text{proj}} \in R^{d_{\text{model}}}$.

Since the self-attention mechanism processes all frames simultaneously, it lacks an inherent understanding of chronological order. To inject sequence structure without altering the data dimensionality, fixed sinusoidal **positional encodings** (PE) are added to the projected embeddings, followed by a **dropout regularization layer**:

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right), \quad PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right) \quad (3)$$

$$X_{\text{input}} = \text{Dropout}(X_{\text{proj}} + PE) \quad (4)$$

where pos represents the frame index within the sequence, and i indicates the channel dimension index.

Transformer Encoder Blocks with Pre-Layer Normalization

The backbone of the architecture consists of a stack of N identical Transformer Encoder blocks. Each block comprises a **Multi-Head Self-Attention** (MHSA) module followed by a Position-Wise Feed-Forward Network (FFN), which uses a Gaussian Error Linear Unit (GELU) activation scaled to a hidden dimension of d_{ff} .

Crucially, the architecture adopts a **Pre-Layer Normalization (Pre-LN)** configuration by enforcing the norm-first paradigm. Unlike traditional Post-LN variants, Pre-LN applies layer normalization immediately before the attention and feed-forward operations rather than on the residual path:

$$X^{(l)'} = X^{(l-1)} + \text{MHSA}(\text{LayerNorm}(X^{(l-1)})) \quad (5)$$

$$X^{(l)} = X^{(l)'} + \text{FFN}(\text{LayerNorm}(X^{(l)'})) \quad (6)$$

This structural adjustment provides a direct, un-obstructed path for gradients to propagate back through the network shortcuts, preventing vanishing or exploding gradients and ensuring stable training convergence. An additional final **LayerNorm** layer is applied to stabilize the encoder's ultimate output representations.

Masked Global Average Pooling and Classification Head

To compress the full temporal sequence dimension (T) down to a singular representation vector suitable for video-level classification, the network implements a custom **Masked Global Average Pooling** layer.

Rather than executing standard unweighted pooling which would skew the classification by absorbing the padding zero-vectors, the final latent representations are multiplied point-wise by a reshaped binary attention mask. The valid feature states are then averaged exclusively over the true, unpadded length of each video sequence:

$$H_{\text{pooled}} = \frac{\sum_{t=1}^T (H_t \cdot M_t)}{\sum_{t=1}^T M_t} \quad (7)$$

where H_t is the encoder output at timestep t and $M_t \in \{0, 1\}$ represents the boolean padding mask.

The resulting pooled vector is fed into a multi-layer classification head consisting of a linear reduction layer to half the hidden dimension size ($d_{\text{model}}/2$), a GELU activation, a dropout layer, and a final linear projection layer to map the features to the definitive corpus class distribution size (`num_classes`):

$$\text{Logits} = \text{Linear}(\text{Dropout}(\text{GELU}(\text{Linear}(H_{\text{pooled}})))) \quad (8)$$

Implementation

Software stack

For the needs of the project, the standard suite of libraries and tools common to machine learning projects was employed. Python served as the base language, with PyTorch providing the framework for defining, training, and evaluating the transformer model. Landmark extraction from video frames was handled by Google’s MediaPipe, specifically its hand and pose estimation tasks, which yield 2D and 3D coordinates for the hands and pose in a single inference pass. The resulting landmark sequences were stored and manipulated as **pandas** DataFrames, while **scikit-learn** supplied preprocessing utilities such as label encoding and the signer-independent train/test split strategy. Standard scientific Python libraries: **NumPy** for numerical operations, **Matplotlib** were used throughout the analysis and evaluation pipeline. Experiment results, including **per-class accuracy breakdowns** and **confusion matrices**, were rendered as PDF reports. Finally, a lightweight interactive demo was built with Streamlit, enabling quick recognition through imported videos without requiring a separate backend server.

Project structure

The project was organised following the Cookiecutter Data Science template [8], a widely adopted convention for reproducible machine learning projects. The top-level directory is divided into purpose-specific subdirectories: **data/** for raw, interim, and processed datasets, **models/** for serialised model checkpoints, and **gsl_detect/** for the installable Python package containing all source code.

The package itself is configured via `pyproject.toml` and built with `flit`, which keeps the build setup minimal and declarative. Common paths and hyperparameters are centralised in `gsl_detect/config.py`, avoiding hardcoded values scattered across scripts.

Version control was managed with Git, with the repository hosted on GitHub. Code quality was enforced throughout development using Ruff, which handled both linting and import sorting, configured via the `ruff` section of `pyproject.toml`.

Challenges and adaptations

Although the initial design accounted for facial landmarks in addition to hand and pose keypoints, experimentation showed that including them degraded classification performance. The face carries little semantic information in sign language beyond what is already captured by the body pose, and in practice the additional coordinates introduced noise without a corresponding gain in accuracy. They were therefore excluded from the final feature vector.

Training was carried out locally rather than on a cloud platform such as Google Colab. Given that the available hardware was sufficient for the task, this avoided the overhead of uploading the dataset to a remote environment and allowed for faster iteration during experimentation.

Results

Hyperparameter optimisation

Several configurations were evaluated before arriving at the final model. The primary axes of exploration were the model dimension (`d_model`), the use of data augmentation, the batch size, and label smoothing.

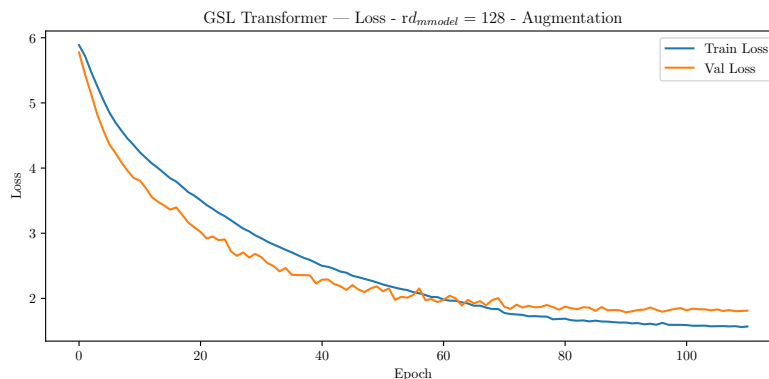


Figure 3: 128 - Augmentation

Figure 3 and Figure 4 compare training and validation loss for `d_model = 128` with and without augmentation respectively. In both cases the model converges within approximately 100 epochs, however the augmented variant exhibits a smaller train-validation gap, indicating better generalisation. Increasing the model dimension to `d_model = 256` (Figure 5) led to faster convergence, reaching a comparable validation loss in roughly 60 epochs, with the augmented configuration

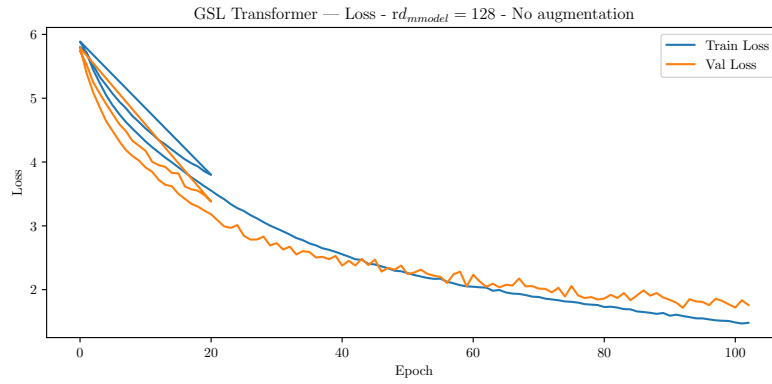


Figure 4: 128 - No augmentation

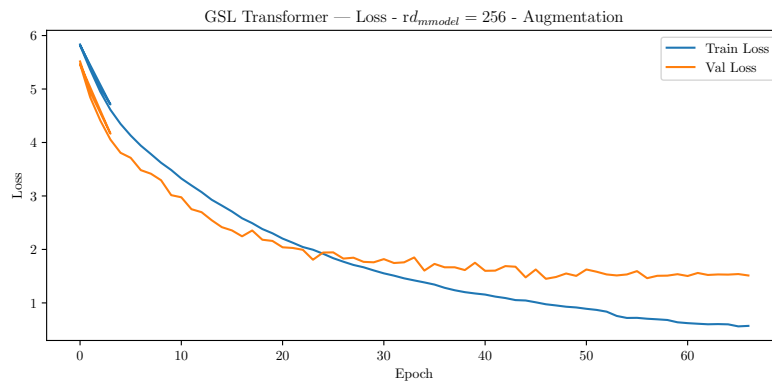


Figure 5: 256 - 32 Batches - No label smoothing

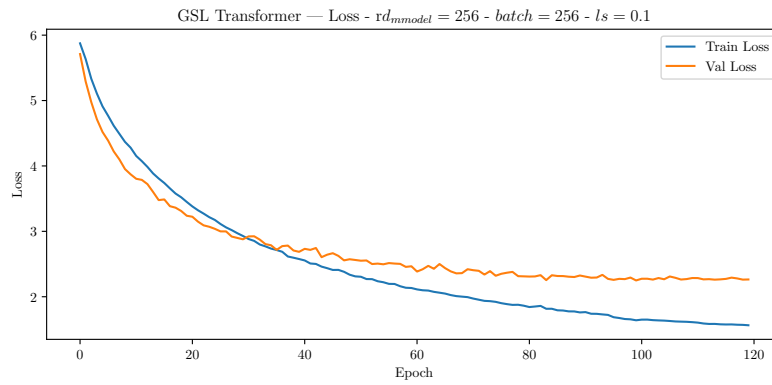


Figure 6: 256 - 64 Batches - Label smoothing



Figure 7: Test set summary

again outperforming its counterpart. A final run with batch size 64 and label smoothing $\epsilon = 0.1$ (Figure 6) produced the best overall result, with the validation loss tracking the training loss more closely throughout training and no sign of overfitting before early stopping triggered. Based on these observations, the final model was trained with `d_model` = 256, 4 attention heads, 3 transformer layers, batch size 64, learning rate 10^{-4} , and label smoothing $\epsilon = 0.1$.

Model performance and convergence

The final model was evaluated on the held-out test set under the signer-independent protocol that was previously discussed. As shown in Figure 7, it achieved a top-1 accuracy of 68.34%, a top-5 accuracy of 93.55%, and a weighted F1 score of 66.91% across 369 classes. The large gap between top-1 and top-5 accuracy suggests that, for a significant portion of misclassified samples, the correct class ranks among the model’s five highest-confidence predictions, pointing to confusion between visually or contextually similar signs rather than complete failure to recognise the gesture.

Convergence was monitored via TensorBoard throughout training. Validation accuracy rose steadily from near zero, crossing 60% around epoch 30 and plateauing in the 66–67% range before early stopping triggered with a patience of 20 epochs. The best checkpoint was saved automatically whenever validation loss improved, ensuring that the evaluated model corresponds to the epoch of lowest generalisation error rather than the last epoch of training.

Error Analysis and Confusion Review

To gain a more granular understanding of the model’s classification behavior, a detailed error analysis was conducted using the final evaluation partition. Figure 8 displays the resulting confusion matrix, while Figure 9 highlights the specific glosses that exhibited the highest rates of mutual misclassification.

An analysis of the most frequently confused glosses reveals that errors were not random; instead, they predominantly occurred between signs sharing significant structural or morphological overlap. When two signs utilize an identical handhape trajectory, the 186-dimensional coordinate

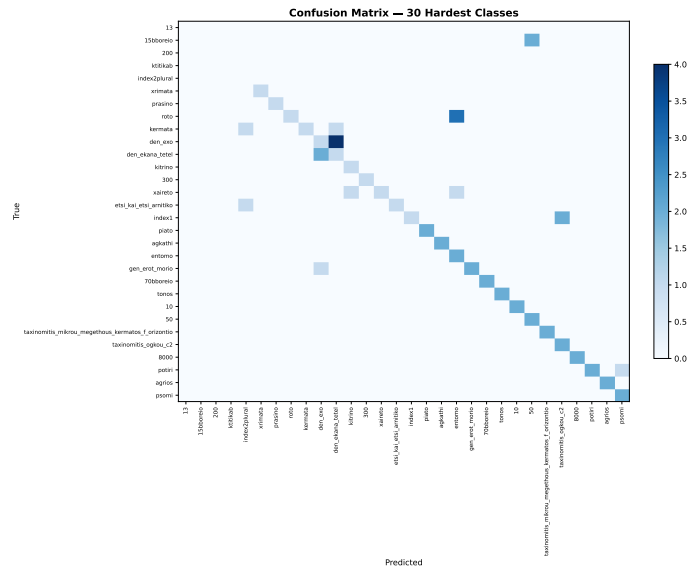


Figure 8: Confusion Matrix for the Sign Language Classification Model.

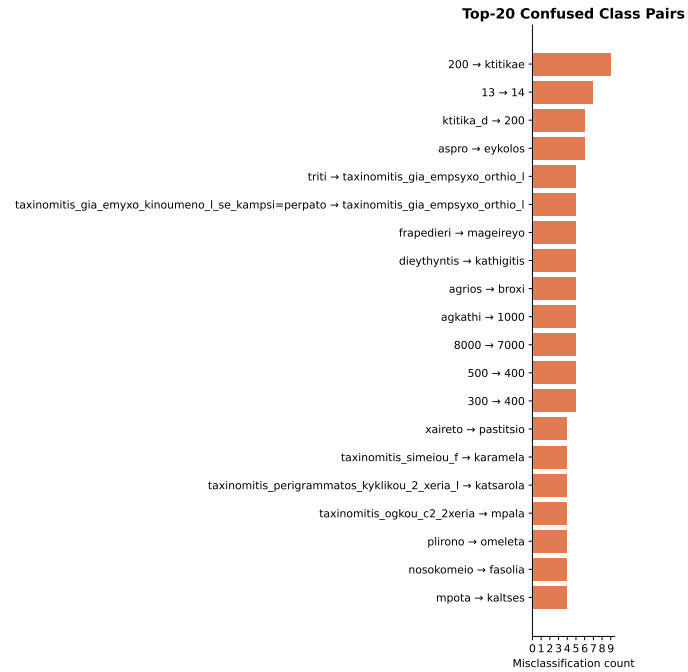


Figure 9: Top confused pairs

feature vector occasionally lacked the spatial resolution required to differentiate them, especially if MediaPipe tracking experienced slight jitter.

Furthermore, a distinct pattern of temporal confusion was identified. Signs that are differentiated primarily by the speed or repetition of a movement (rather than the spatial path itself) showed higher error rates. This suggests that while the Transformer’s self-attention mechanism successfully mapped global sequence structures, it occasionally struggled to resolve fine-grained temporal boundaries in highly dynamic, rapid signing sequences.

Demonstration application

A lightweight demonstration application was developed using Streamlit, providing a graphical interface for interacting with the trained model without requiring any additional backend infrastructure. The application offers two modes of operation.

The first mode accepts a pre-recorded video file in a common format (.mp4, .avi, .mov). The uploaded clip is written to a temporary file, processed frame-by-frame through the MediaPipe landmark extractor, and the resulting sequence is normalised, padded, and passed to the transformer model. The predicted gloss is then displayed to the user.

The second mode allows the user to sign directly in front of a webcam. Frames are captured continuously and landmark coordinates are accumulated into a rolling buffer. Once the buffer reaches a fixed length of 90 frames, the sequence is normalised and fed to the model, producing a prediction that is displayed on screen. The buffer is then cleared and accumulation begins again for the next sign.

In both modes, the model and dataset metadata are initialised once at startup using Streamlit’s `@st.cache_resource` decorator, ensuring that the checkpoint is loaded and the label encoder is available for the duration of the session without repeated disk reads.

Appendix: Accuracy curves for different model configurations

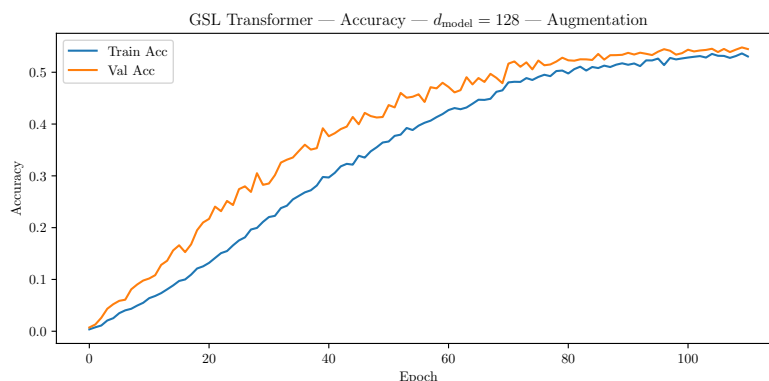


Figure 10: 128 - 32 - No label smoothing - No augmentation



Figure 11: 128 - 32 - No label smoothing - Augmentation

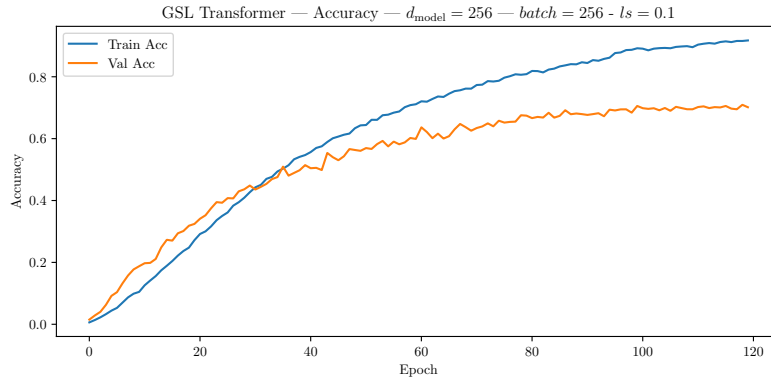


Figure 12: 256 - 32 - No Label smoothing - Augmentation

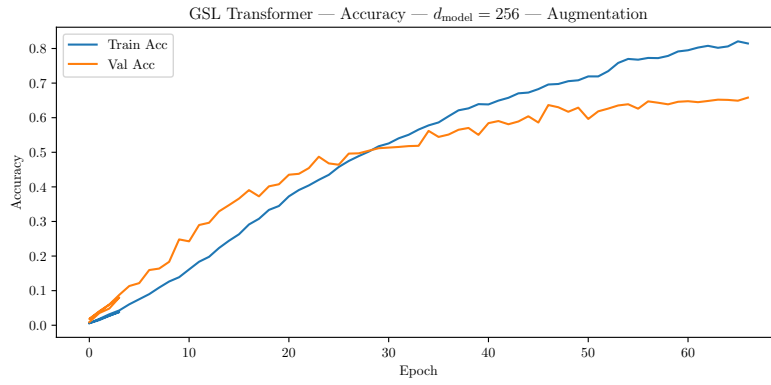


Figure 13: 256 - 64 - Label smoothing - Augmentation

Conclusion

This work presented a transformer-based classifier for isolated Greek Sign Language recognition, operating directly on body and hand landmark sequences extracted by MediaPipe. Without relying on raw video or appearance features, the model achieved 68.34% top-1 and 93.55% top-5 accuracy across 369 gloss classes under a signer-independent evaluation protocol, demonstrating that skeleton-based representations carry sufficient discriminative information for large-vocabulary GSL recognition.

The main limitation of the current system is its restriction to isolated sign classification. Real-world communication involves continuous signing, co-articulation between signs, and facial grammar that was deliberately excluded here. The demonstration application developed alongside the model provides a concrete starting point for extending the system toward real-time continuous recognition.

References

- [1] Naveen Ghorpade, Ajay Sudhir Bale, Raghavendran T S, Sumit U, Swaraj Sahani, and Manjunath K B. Real-time interpretation of sign language with custom gestures and multi-output. In *2025 International Conference on Information, Implementation, and Innovation in Technology (I2ITCON)*, pages 1–6, 2025.
- [2] Yazan Alahmed, Ammar Al-Hawamdeh, and Mohammed Jassim Al Ansari. Ai-powered sign language translation: Enhancing communication with mediapipe and machine learning. In *2025 2nd International Generative AI and Computational Language Modelling Conference (GACLM)*, pages 349–356, 2025.
- [3] Andrei Enachi, Corneliu-Octavian Turcu, Bogdan-Constantin Sghera, and Andrei-Gabriel Ungureanu. Real-Time Sign Language Interpretation: Integrating Gesture Recognition with Syntactic Analysis. In *2025 25th International Conference on Control Systems and Computer Science (CSCS)*, Suceava, Romania, 2025. IEEE.
- [4] Andreas Voskou, Konstantinos P. Panousis, Harris Partaourides, Kyriakos Toliass, and Sotirios Chatzis. A new dataset for end-to-end sign language translation: The greek elementary school dataset. In *2023 IEEE/CVF International Conference on Computer Vision Workshops (ICCVW)*, pages 1958–1967, 2023.
- [5] Nikolas Adaloglou, Theocharis Chatzis, Ilias Papastratis, Andreas Stergioulas, Georgios Th Papadopoulos, Vassia Zacharopoulou, George J Xydopoulos, Klimnis Atzakas, Dimitris Papazachariou, and Petros Daras. A comprehensive study on sign language recognition methods. *arXiv preprint arXiv:2007.12530*, 2020.
- [6] K. Papadimitriou, G. Sapountzaki, K. Vasilaki, E. Efthimiou, S.-E. Fotinea, and G. Potamianos. SL-REDU GSL: A large greek sign language recognition corpus. In *Proc. IEEE International Conference on Acoustics, Speech and Signal Processing Workshop on Sign Language Translation and Avatar Technology (ICASSPW-SLTAT)*, pages 1–5, 2023.
- [7] K. Papadimitriou, Gerasimos Potamianos, Galini Sapountzaki, et al. Greek sign language recognition for an education platform. *Universal Access in the Information Society*, 24(1):51–68, 2025.
- [8] DrivenData. Cookiecutter data science. <https://cookiecutter-data-science.drivendata.org/>, 2023. Accessed: 2025-03-01.
- [9] Kokkinou Kleopatra Tsougriani Ioanna. The Influence of Greek Sign Language in Writing of the Deaf and Hard of Hearing People. Undergraduate thesis, Technological Educational Institute of Western Greece, School of Health and Welfare Professions, Department of Speech and Language Therapy, Patras, Greece, 2019. Text in Greek.
- [10] Sadman Hasib Emon, Antu Roy Chowdhury, Md Abu Ismail Siddique, Md. Mehedi Hassan, Arif Hossen, and Md. Siyam Sade. Real-time detection and translation of bangla sign language characters using deep learning. In *2025 International Conference on Quantum Photonics, Artificial Intelligence, and Networking (QPAIN)*, pages 1–6, 2025.

- [11] Muthuraju V, Nalini N, and Rashmi M. American sign language translation with emotion recognition. In *2025 IEEE 4th International Conference for Advancement in Technology (ICONAT)*, pages 1–5, 2025.